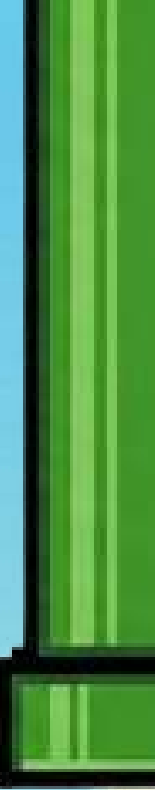
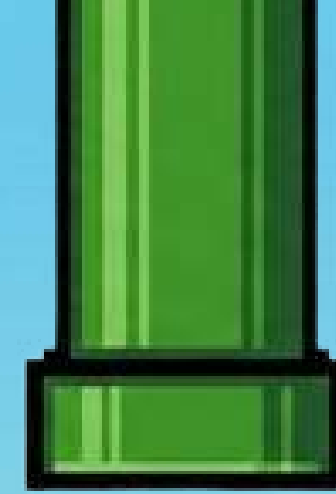


PROJECT GAME -FLAPPY BIRD

SESSION 2-Final



Step 1

```
from turtle import Turtle
import random

class PipeManager:
    def __init__(self):
        self.pipes = []
        self.pipe_speed = 3
        self.counter = 0
```

Di Pipe.py

Class PipeManager berfungsi untuk mengatur semua pipa dalam permainan. Variabel `self.pipes = []` digunakan untuk menyimpan daftar pipa yang muncul di layar. `self.pipe_speed = 3` menentukan kecepatan pergerakan pipa ke kiri, sedangkan `self.counter = 0` digunakan sebagai penghitung untuk mengatur kapan pipa baru harus dibuat. Dengan class ini, pipa dapat dibuat, disimpan, dan digerakkan secara teratur selama permainan berlangsung.

Step 2

```
def create_pipe(self): 1 usage
    self.counter += 1

    if self.counter % 110 == 0:
        gap_y = random.randint(-80, 80)

        # PIPA ATAS
        top_pipe = Turtle()
        top_pipe.shape("assets/pipe1.gif")
        top_pipe.penup()
        top_pipe.goto(x: 330, gap_y + 330)

        # PIPA BAWAH
        bottom_pipe = Turtle()
        bottom_pipe.shape("assets/pipe2.gif")
        bottom_pipe.penup()
        bottom_pipe.goto(x: 330, gap_y - 290)
```

Di Pipe.py

Method `create_pipe()` berfungsi untuk membuat pasangan pipa atas dan pipa bawah secara otomatis. Setiap counter mencapai kelipatan 110, program membuat pipa baru dengan posisi celah (`gap_y`) yang dipilih secara acak agar permainan lebih menantang.

Pipa atas dibuat menggunakan gambar `pipe1.gif` dan ditempatkan di bagian atas layar, sedangkan pipa bawah menggunakan `pipe2.gif` dan ditempatkan di bagian bawah. Kedua pipa memiliki jarak tertentu sebagai celah yang harus dilewati burung. Dengan method ini, rintangan baru akan terus muncul selama permainan berlangsung.



Step 3

Di Pipe.py

```
self.pipes.append({
    "top": top_pipe,
    "bottom": bottom_pipe,
    "gap_y": gap_y,
    "scored": False
})

def move_pipes(self):
    for pipe_pair in self.pipes:
        pipe_pair["top"].setx(pipe_pair["top"].xcor() - self.pipe_speed)
        pipe_pair["bottom"].setx(pipe_pair["bottom"].xcor() - self.pipe_speed)
```

Bagian `self.pipes.append({...})` digunakan untuk menyimpan data pipa yang baru dibuat ke dalam daftar `pipes`. Data yang disimpan meliputi pipa atas (`top`), pipa bawah (`bottom`), posisi celah (`gap_y`), dan status `scored` yang bernilai `False` karena pemain belum mendapatkan poin dari pipa tersebut.

Method `move_pipes()` berfungsi untuk menggerakkan semua pipa ke kiri. Program melakukan perulangan pada setiap pasangan pipa, lalu mengurangi posisi X pipa atas dan bawah sebesar `pipe_speed`. Dengan cara ini, pipa terlihat bergerak dari kanan ke kiri sebagai rintangan yang harus dilewati burung.



Step 4

```
from turtle import Screen
from bird import Bird
from pipe import PipeManager
from scoreboard import Scoreboard
import time

screen = Screen()
screen.setup(width=600, height=700)
screen.title("Flappy Bird Python")

screen.bgpic("assets/background.gif")

screen.register_shape("assets/bird.gif")
screen.register_shape("assets/pipe2.gif")
screen.register_shape("assets/pipe1.gif")

screen.tracer(0)
```

Di Main.py

Bagian ini merupakan program utama yang mengatur tampilan dan jalannya game. Program mengimpor class Bird, PipeManager, dan Scoreboard, serta modul time untuk mengatur kecepatan permainan.

Objek Screen() digunakan untuk membuat jendela game dengan ukuran 600 × 700 piksel dan judul "Flappy Bird Python". Kemudian bgpic() digunakan untuk memasang gambar latar belakang agar tampilan game lebih menarik.

Perintah register_shape() mendaftarkan gambar burung dan pipa sehingga dapat digunakan sebagai bentuk objek dalam game. Terakhir, screen.tracer(0) mematikan pembaruan layar otomatis agar animasi menjadi lebih halus dan dapat dikontrol secara manual menggunakan screen.update().



Step 5

Di Main.py

```
bird = Bird()
pipes = PipeManager()
scoreboard = Scoreboard()

screen.listen()
screen.onkey(bird.jump, key="space")

game_is_on = True

while game_is_on:
    time.sleep(0.02)
    screen.update()

    bird.move()
    pipes.create_pipe()
    pipes.move_pipes()

    for pipe_pair in pipes.pipes:
        pipe_x = pipe_pair["top"].xcor()

        if not pipe_pair["scored"] and pipe_x < bird.xcor():
            pipe_pair["scored"] = True
            scoreboard.add_score()
```

Bagian ini membuat objek burung (Bird), pengelola pipa (PipeManager), dan papan skor (Scoreboard). Program juga mengaktifkan input keyboard dengan `screen.listen()` dan menghubungkan tombol spasi ke method `bird.jump()`, sehingga burung dapat melompat saat pemain menekan spasi.



Selama `game_is_on` bernilai `True`, game akan terus berjalan dalam perulangan. Pada setiap putaran, layar diperbarui, burung bergerak, pipa baru dibuat jika diperlukan, dan semua pipa digerakkan ke kiri.

Program juga memeriksa apakah burung telah berhasil melewati pipa. Jika pipa belum dihitung (`scored = False`) dan posisi pipa sudah berada di belakang burung, maka skor bertambah 1 dan status pipa diubah menjadi `True` agar tidak dihitung lagi. Dengan demikian, pemain memperoleh poin setiap kali berhasil melewati satu pasangan pipa.

Step 6

Di Main.py

```
for pipe_pair in pipes.pipes:
    pipe_x = pipe_pair["top"].xcor()
    gap_y = pipe_pair["gap_y"]

    if pipe_x - 35 < bird.xcor() < pipe_x + 35:
        if bird.ycor() > gap_y + 95 or bird.ycor() < gap_y - 95:
            game_is_on = False
            scoreboard.game_over()

    if bird.ycor() < -330 or bird.ycor() > 330:
        game_is_on = False
        scoreboard.game_over()

screen.exitonclick()
```

Bagian ini berfungsi untuk mendeteksi tabrakan (collision) dan menentukan kapan permainan berakhir. Program memeriksa setiap pasangan pipa untuk melihat apakah posisi burung berada sejajar dengan pipa pada sumbu X.

Jika burung berada di area pipa, program akan mengecek apakah posisi Y burung masih berada di dalam celah yang aman. Jika burung menabrak pipa atas atau pipa bawah, maka `game_is_on` diubah menjadi `False` dan method `game_over()` dipanggil untuk menampilkan pesan akhir permainan.

Selain itu, program juga memeriksa apakah burung keluar dari batas layar. Jika burung jatuh terlalu bawah ($y < -330$) atau terbang terlalu tinggi ($y > 330$), permainan langsung dihentikan dan layar GAME OVER ditampilkan.

Terakhir, `screen.exitonclick()` membuat jendela game tetap terbuka setelah permainan selesai dan baru akan ditutup ketika pengguna mengklik layar

Step 7

Di Main.py

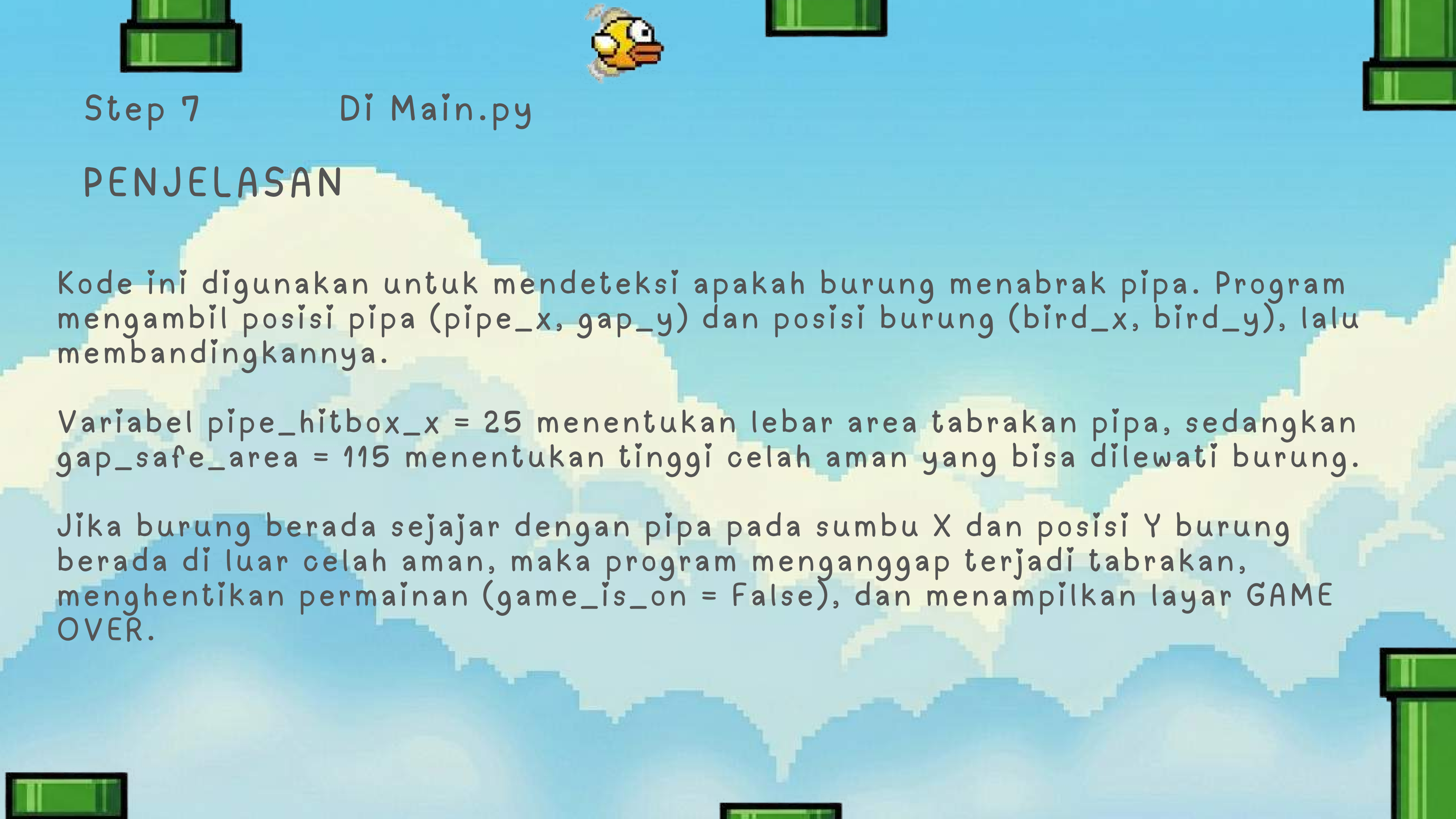
Sekarang Ada Bug!!!! Burung Tidak kena Pipe Tapi game over!!!

```
for pipe_pair in pipes.pipes:
    pipe_x = pipe_pair["top"].xcor()
    gap_y = pipe_pair["gap_y"]

    bird_x = bird.xcor()
    bird_y = bird.ycor()

    # hitbox dibuat lebih kecil agar tidak terlalu sensitif
    pipe_hitbox_x = 25
    gap_safe_area = 115

    if pipe_x - pipe_hitbox_x < bird_x < pipe_x + pipe_hitbox_x:
        if bird_y > gap_y + gap_safe_area or bird_y < gap_y - gap_safe_area:
            game_is_on = False
            scoreboard.game_over()
```

Step 7

Di Main.py

PENJELASAN

Kode ini digunakan untuk mendeteksi apakah burung menabrak pipa. Program mengambil posisi pipa (`pipe_x`, `gap_y`) dan posisi burung (`bird_x`, `bird_y`), lalu membandingkannya.

Variabel `pipe_hitbox_x = 25` menentukan lebar area tabrakan pipa, sedangkan `gap_safe_area = 115` menentukan tinggi celah aman yang bisa dilewati burung.

Jika burung berada sejajar dengan pipa pada sumbu X dan posisi Y burung berada di luar celah aman, maka program menganggap terjadi tabrakan, menghentikan permainan (`game_is_on = False`), dan menampilkan layar GAME OVER.

THANK
YOU



